

2017-3(a)

Integrity Constraint: set of rules that are applied to the relations and relationships to ensure that the overall validity, integrity, consistency of the data is maintained.

1. Domain Integrity Constraint: set of rules that restrict the kinds of values a column hold in a table.

2. Entity Integrity Constraint: used to ensure the uniqueness of ^{Key} each record in a table. ^{unique, primary key, not null, check}

^{restriction on duplicate values} used to identify row.

3. Referential Integrity Constraint: ensures the relationships between ~~two~~ tables are valid and that foreign key values in one table match primary key values in another table/null.

Entity Integrity

4. Key Constraint:

- each row uniquely identifiable through primary key.
- guarantees no primary key is null/duplicated.

Key

→ primary key, foreign key, unique ensure the uniqueness and non null property of a specific set of columns.

Entity Integrity:

ensure the validity and uniqueness of primary key of a table.

[2017-3(b)]

Trigger is a special type of stored procedure that is automatically executed in response to certain database events. The purpose of a trigger is to perform specific action(s) based on the occurrence of the triggering event.

~~Create trigger~~

~~on delete~~

~~after delete on accounts
for each row~~

~~declare~~

~~cursor c is select account_no from accounts~~

~~where~~

~~join depositor using
(account_no)~~

account(a-no, balance) ✓

→ depositor(c-id, ~~e-name~~ a-no)

create trigger kill

after delete on account

for each row

declare

cnt number;

begin

into cnt

select count(*) from depositor where

c-id = (select c-id from depositor where

a-no = :old.a-no);

if cnt = 0 then

delete from depositor where c-id =

(select c-id from depositor where a-no = :old.a-no);

endif;

end;

2017_4(a)

Attribute closure α^+ is defined as the set of attributes that are functionally determined by α under F .

Uses:

(i) Testing superkey:

$$R \subseteq \alpha^+ \quad \alpha - \text{superkey}$$

(ii) Testing functional dependency:

$$\alpha \rightarrow \beta$$

$$\beta \subseteq \alpha^+ \rightarrow (\alpha \rightarrow \beta) \text{ holds}$$

(iii) Computing F^+ :

For each $\gamma \subseteq R$ find γ^+

$$\text{for each } \delta \subseteq \gamma^+, \boxed{\gamma \rightarrow \delta}$$

2017-4(b)

BCNF - $\alpha \rightarrow \beta$ ① trivial ② α superkey
 ③ each attribute A in $\beta - \alpha$ contained in candid key of R.

3NF > BCNF $\rightarrow$ preserve dependency by allowing some redundant attributes (both lossless decomposition).

Book(title, author-name, author-designation, type, price, publisher).

⊗ title $\rightarrow$ publisher, type violates ①, ②

R_1 (title, publisher, type) Book(title, author-name, author-designation, price).
 BCNF

⊗ type $\rightarrow$ price ⊗

⊗ author-name $\rightarrow$ author-designation. ①, ②

$R_2 = (\text{title}, \text{author_name}, \text{price})$

BCNF

$R_3 = (\text{author_name}, \text{author_designation})$

BCNF.

monif (purov) pvo hoo/or) to (vobv) pvo tge

2017-4(c)

(hrombrop)

mon-tge hoo

Structured data: tge wot-tge mon-

→ highly organized and follows a over

predefined data model/schema elements

are addressable for analysis.

→ based on relational database table.

→ less flexible

= mon-tge roborv over notvobv

Student

ID	Name
10	Nihal

(hrombrop)

Semi ~~un~~ structured data ~~is not~~ is

doesn't have fixed schema but possesses some level of organization that make it easier to analyze.

→ based on XML/RDF/JSON

→ more flexible than structured data

```
{ "name": "Nihal",  
  "address": {  
    "street": "123",  
    "city": "ABC"
```

```
},
```

```
"phone": [
```

```
  { "type": "home",  
    "number": "123"
```

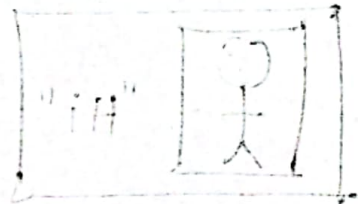
```
},
```

```
{
```

```
  ==
```

```
},
```

```
],
```



```
"hobbies": ["—", "—", "—"]
```

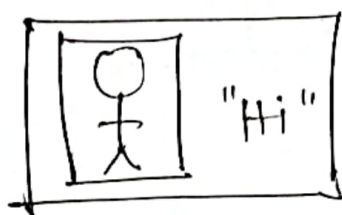
```
}
```

Unstructured data

data with no predefined organizational form / specific format.

→ based on character / binary data

→ flexible



"John" "Mum"

"123456789"

"ABC" : "DEF"

"123456789" : "123456789"

{

name : "John"

phone : "123456789"

number : "123456789"

}

}

}

2017_2(d)

	max	min
no. of pointers in <u>leaf</u> node	n	$\lceil \frac{n}{2} \rceil$
no. of values in leaf/non leaf node	n-1	$\lceil \frac{n-1}{2} \rceil$

x-keys

$$x \times 9 + (x+1) \times 7 \leq 512$$

$$\Rightarrow 9x + 7x + 7 \leq 512$$

$$\Rightarrow 16x \leq 505$$

$$\Rightarrow x \leq 31.56$$

$$x \approx 31. \text{ (A)}$$